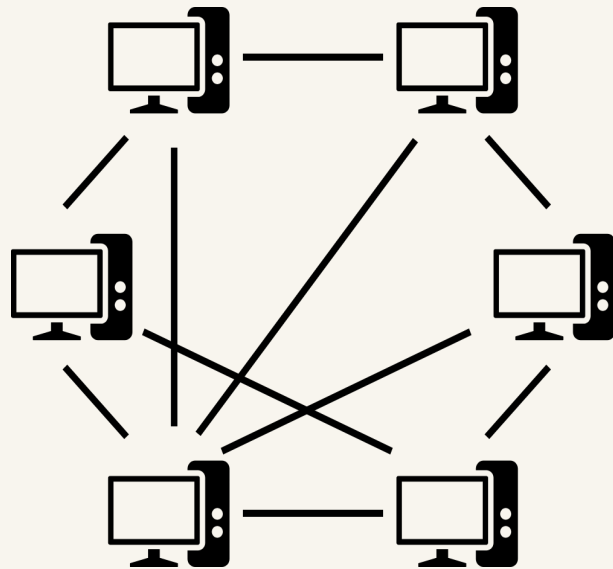


Kademlia: A Peer-to-Peer System Designed from a Single Principle

How a novel XOR-based metric topology creates a provably consistent and performant Distributed Hash Table.



Team Members

Rithvik Jami CS23BTECH11022

Akshaya Tharun CS23BTECH11003

Sai Rishi Kasturi CS23BTECH11024

Jakkala Naga Rohith Kumar CS23BTECH11021

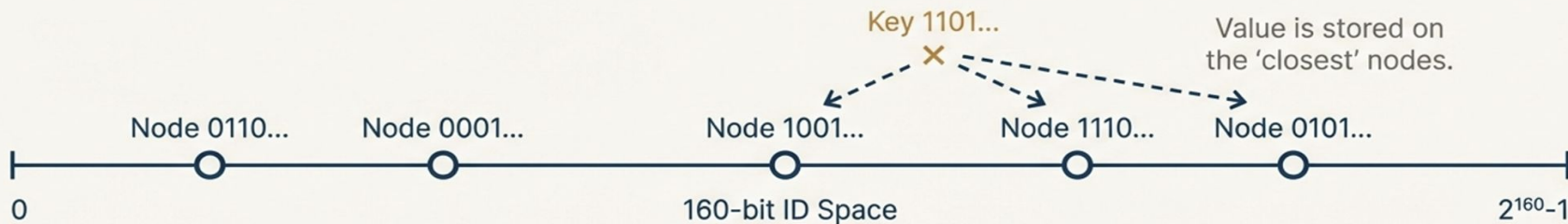
Muvvala Sairam Suhas CS23BTECH11038

Ruchika Thakur Singh CS23BTECH11061

The Foundational Principles of a Distributed Hash Table

Kademlia builds upon the standard DHT model, which relies on several core principles:

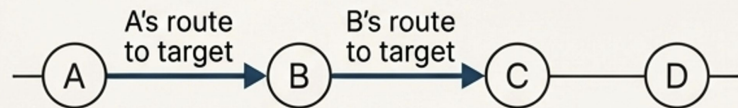
- **Shared Address Space:** The system uses a 160-bit space for both node IDs and keys.
- **Node IDs:** Participating computers are each assigned a unique 160-bit node ID.
- **Keys:** Data is identified by a 160-bit key, likely opaque .
- **The Principle of Closeness:** <key,value> pairs are stored on nodes with IDs that are 'close' to the key.
- **Efficient Routing:** A routing algorithm must exist to allow any node to efficiently locate the nodes responsible for any given key.



The Power of a Symmetric Topology

The choice of XOR is not arbitrary; its properties create a fundamentally more efficient and robust network topology.

Asymmetric (e.g., Chord)

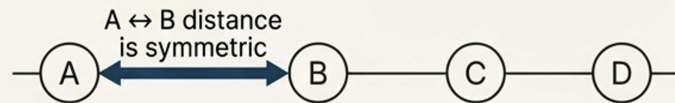


Consequence: Messages from an incoming query do not provide useful routing information for the recipient. A learns about B, but B learns nothing useful about A.

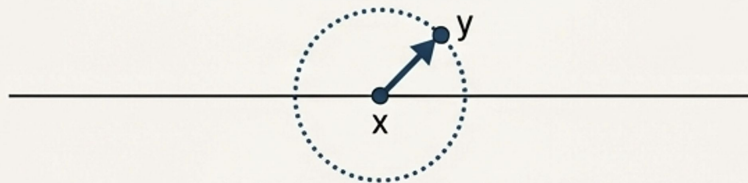
Unidirectionality

For any point x and distance $\Delta > 0$, there is exactly one point y such that $d(x, y) = \Delta$.

Symmetric (Kademlia)



Symmetry ($d(x, y) = d(y, x)$): Every message exchanged—request or reply—conveys or reinforces useful routing information for both parties.



A Novel Metric: Defining Distance as XOR

The core innovation of Kademlia is its metric for distance. The distance between two 160-bit identifiers, x and y , is defined as their bitwise exclusive or (XOR), interpreted as an integer.

$$d(x, y) = x \oplus y$$

$x =$	1	0	1	1	0	1	0	1
$y =$	0	1	1	0	0	1	1	1
$x \oplus y =$	1	1	0	1	0	0	1	0

XOR satisfies the necessary mathematical properties of a valid metric:

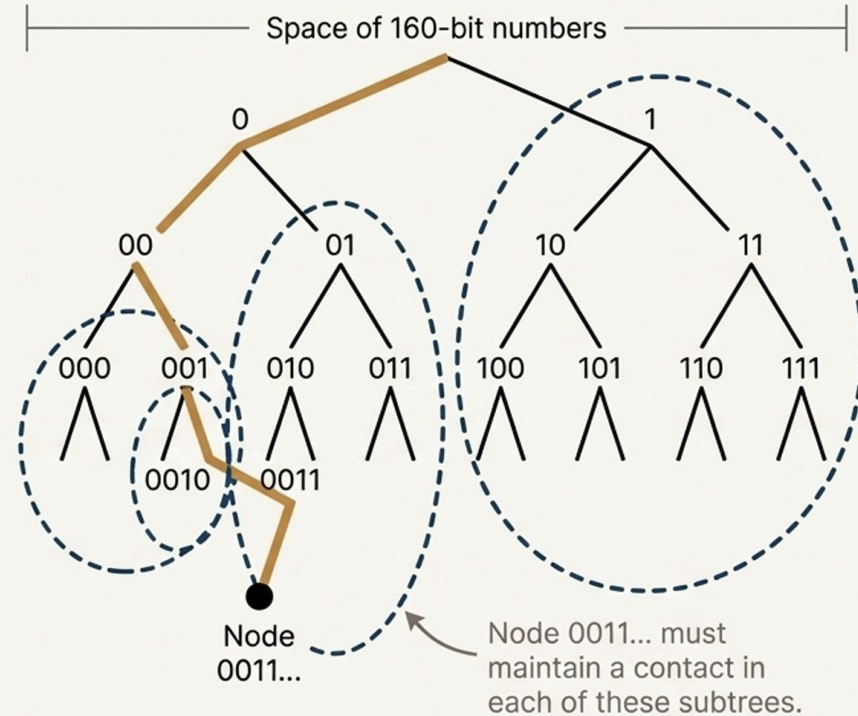
- $d(x, x) = 0$
- $d(x, y) > 0$ if $x \neq y$
- Symmetry: $d(x, y) = d(y, x)$
- Triangle Inequality: $d(x, z) \leq d(x, y) + d(y, z)$

The XOR Metric Organises the Network into a Binary Tree

The XOR distance metric is not just a number; it imposes a structure on the entire network.

Kademlia treats nodes as leaves in a binary tree, where a node's position is determined by the shortest unique prefix of its ID.

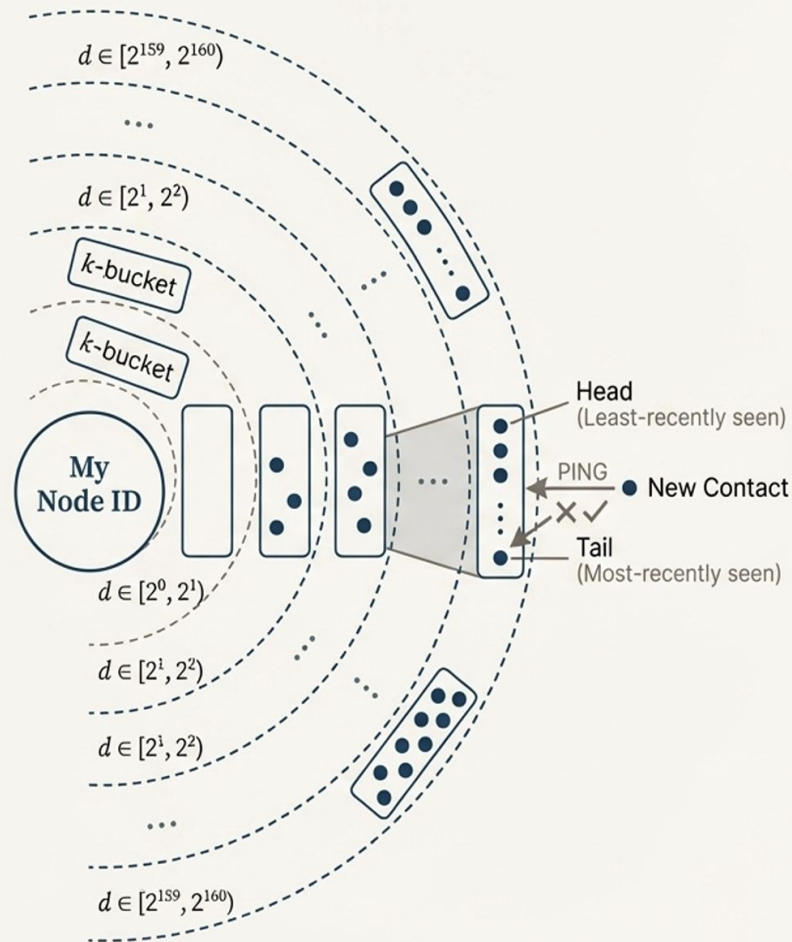
- **Subtree Division:** For any given node, the tree is divided into a series of successively smaller subtrees that do not contain the node.
- **Routing Guarantee:** The protocol ensures every node knows at least one other node in each of its subtrees (if occupied).
- **Navigability:** This guarantee is the foundation of the lookup algorithm.



'k-buckets'. A Node's View of the Network

A node's knowledge is managed in lists called 'k-buckets'.

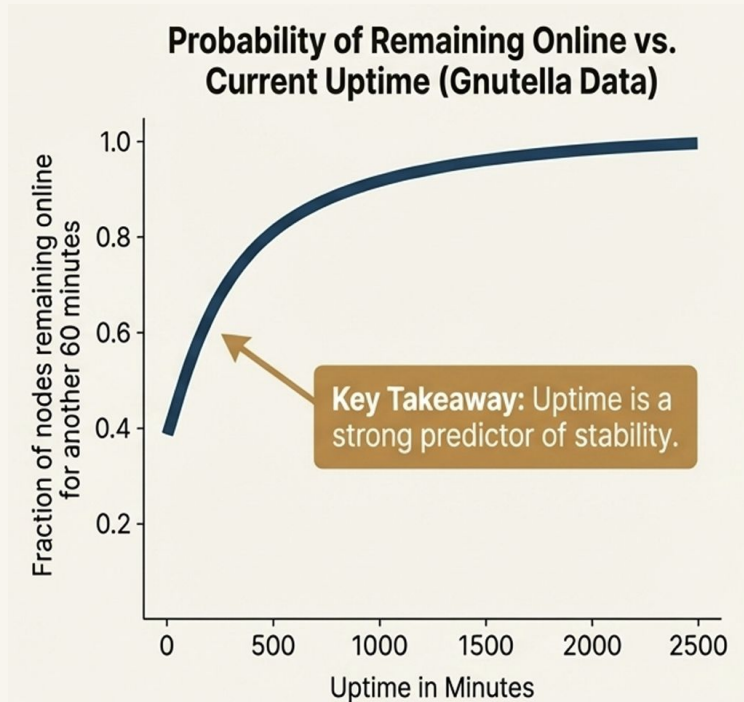
- **Structure:** For each i from 0 to 159, a node maintains a bucket for nodes with distance in the range $[2^i, 2^{i+1})$
- **Content:** Each bucket stores up to k triples of $\langle \text{IP address, UDP port, Node ID} \rangle$.
- **Update Mechanism:** When a message is received from a node:
 - If sender exists, move it to the tail (most-recently seen).
 - If not, and bucket has $< k$ entries, add sender to the tail.
 - If bucket is full, ping the least-recently seen node (at the head) and add the new sender.



Preferring Stability: An Eviction Policy Based on Uptime

The '**k-bucket**' eviction strategy (pinging the oldest contact before discarding it) is a crucial design choice. It prioritises long-lived nodes over new ones.

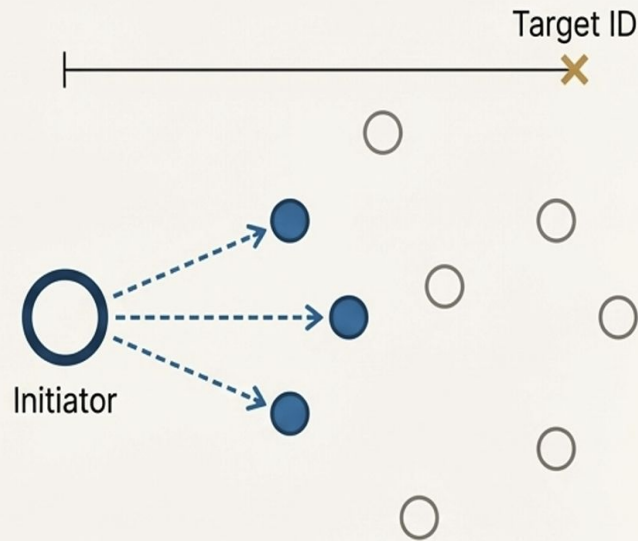
- **The Rationale:** Analysis of real-world P2P networks shows that a node's uptime is a strong predictor of its future availability.
- **The Benefits:**
 1. **Stability:** Maximises the probability that nodes in routing tables will remain online.
 2. **DoS Resistance:** Prevents an attacker from flooding routing state by flooding the system with new, short-lived nodes.



Finding a Node: The Parallel, Recursive Lookup

The most important procedure is locating the k closest nodes to a target ID. Kademlia uses a recursive algorithm initiated with parallel requests.

- **The RPC:** A FIND_NODE RPC takes a 160-bit ID argument. The recipient returns the k nodes it knows that are closest to that target.
- **Step 1:** Initiation: The initiator selects α nodes from its closest non-empty k -bucket.
- **Step 2:** Parallel Query: The initiator sends parallel, asynchronous FIND_NODE RPCs to these α nodes simultaneously (α is a concurrency parameter, e.g., 3).

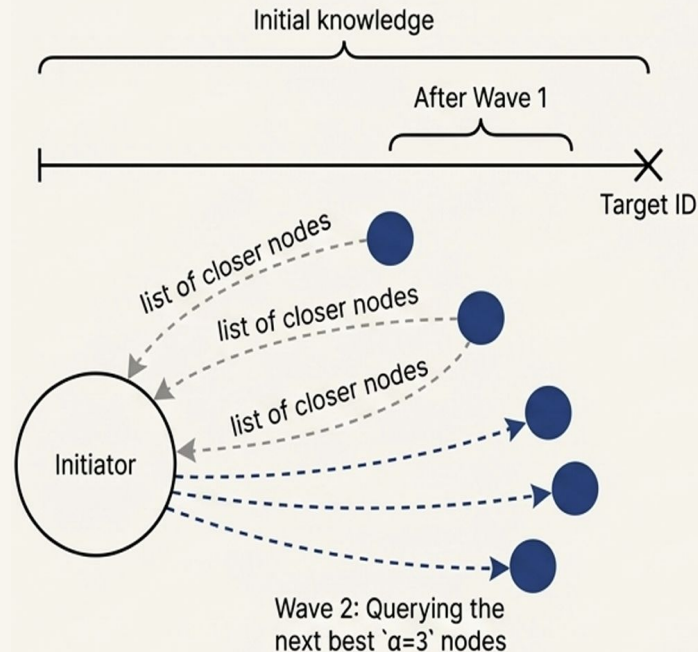


Wave 1: Parallel FIND_NODE RPCs to ' $\alpha=3$ ' closest known nodes

Converging on the Target with Asynchronous Recursion

The lookup continues by iteratively contacting closer and closer nodes.

- **Iterative Step:** From the results, the initiator picks the α best nodes (closest to the target) it has not yet queried and sends them new FIND- NODE RPCs.
- **Termination:** The lookup terminates when the initiator has queried and received responses from the k closest nodes it has seen.
- Key Advantages of $\alpha > 1$:
 - **Low Latency:** Choose routes based on network latency.
 - **Fault Tolerance:** Route around failed nodes without waiting for timeouts.



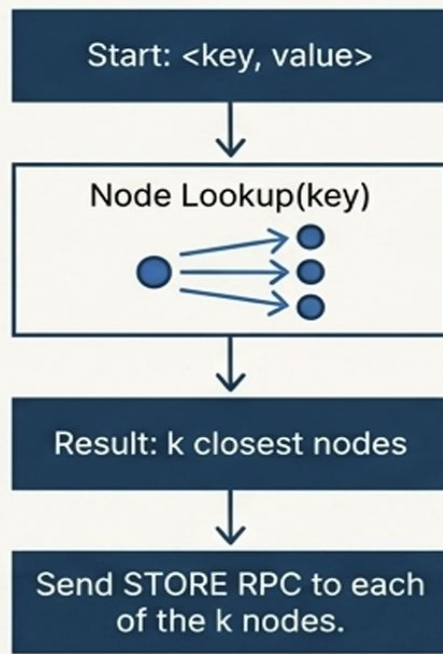
Iterative lookup code

```
std::vector<NodeInfo> Node::iterative_find_node(const NodeID &target) {
    auto candidates = _rt->find_closest(target, K_BUCKET_SIZE);
    std::unordered_set<std::string> queried;
    bool progress = true;
    while (progress) {
        progress = false;
        size_t q = 0;
        for (size_t i = 0; i < candidates.size() && q < ALPHA; ++i) {
            if (queried.count(candidates[i].addr)) continue;
            queried.insert(candidates[i].addr);
            q++;
            auto res = rpc_find_node(candidates[i].addr, target);
            if (res) {
                for (auto &n : res->nodes) {
                    bool exists = false;
                    for (auto &c : candidates) if (c.addr == n.addr) { exists = true; break; }
                    if (!exists) { candidates.push_back(n); progress = true; }
                }
            }
        }
        std::sort(candidates.begin(), candidates.end(), [&](const NodeInfo &a, const NodeInfo &b) {
            NodeID da = a.id ^ target, db = b.id ^ target; return da.less_than(db);
        });
        if (candidates.size() > K_BUCKET_SIZE) candidates.resize(K_BUCKET_SIZE);
    }
    return candidates;
}
```

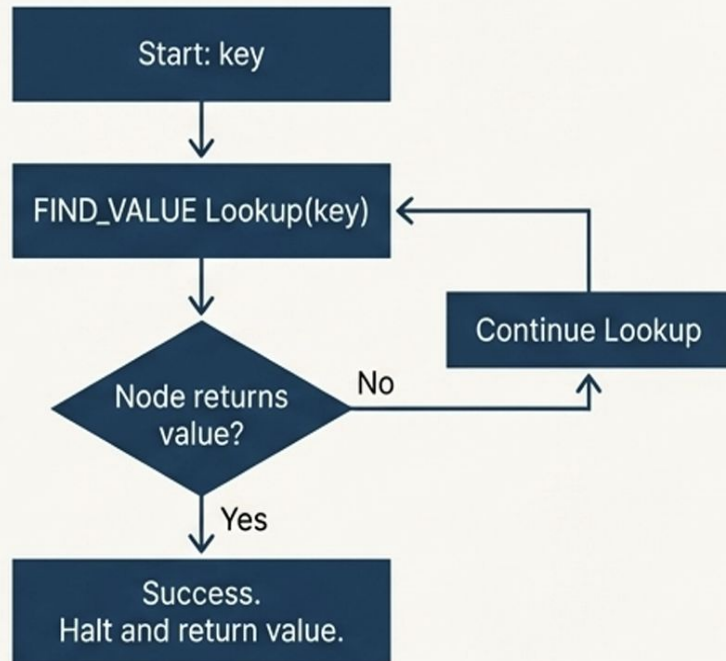
Lookups are the Foundation for Storing and Finding Values

Most core operations are simple applications of the node lookup procedure

Storing Data (STORE)



Finding Data (FIND_VALUE)



Code Snippets of STORE and FIND_VAL rpc's

```
bool Node::store_value(
    const NodeID &key_id,
    const std::string &key_hex,
    const std::string &value)
{
    // 1. Find k-closest peers
    auto closest = iterative_find_node(key_id);

    // 2. Send STORE RPCs
    for (auto &peer : closest) {
        rpc_store(peer.addr, key_hex, value);
    }
    return true;
}
```

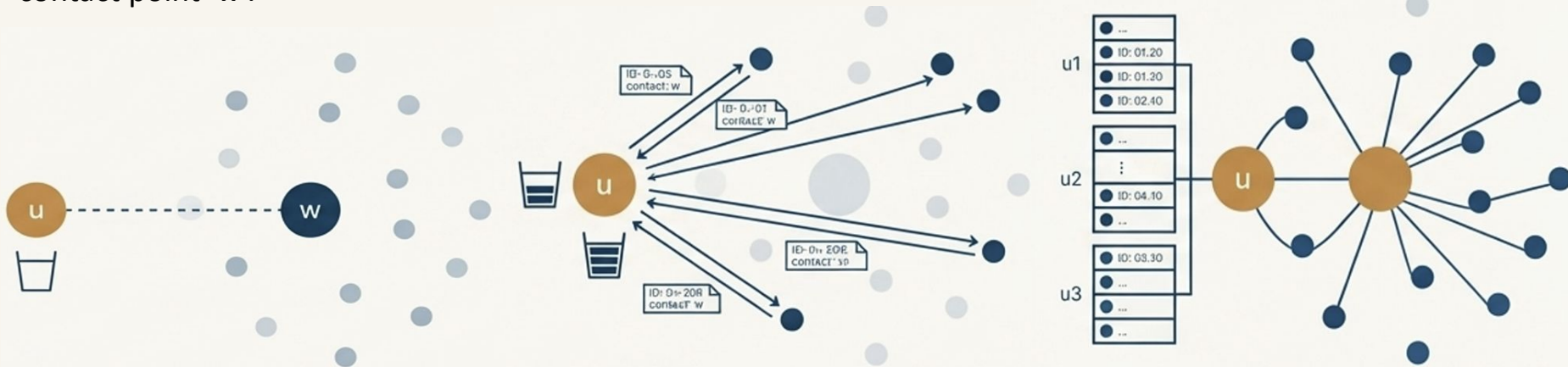
```
std::variant...>
Node::iterative_find_value(
    const std::string &key_hex,
    const NodeID &key_id)
{
    // ... same loop as find_node ...
    auto res = rpc_find_value(/* ... */);
    if (!res) continue;
    if (res->value.has_value()){
        return *res->value; // Found it!
    }
    // Keep searching...
    for (auto &n : res->nodes) {
        // ... add to candidates ...
    }
}
```

Some Implementation Details

- Each node uses a separate thread receiver for receiving.
- UDP network is used by 2 nodes in a network to communicate using a protocol(a simple delimiter for serializing and deserializing the messages in our case) for different rpc(remote procedure calls) .
- Each node has mainly two data structures
 - Routing_table
 - Data (KEY - VALUE)
- Put and get functions are used to store and access information in the network.
- Command line is used by a user to enter the p2p network. Each successive user bootstraps some other existing user in the network to join the network.
- Deque is used to implement k-buckets for efficient eviction and relocation appropriately based on LRU(Least Recently Used) policy.

Joining the Network: A Simple Bootstrap Process

A new node 'u' can join with minimal configuration, needing only a single contact point 'w'.



1. Initial Contact: Node 'u' adds contact 'w' to a k-bucket'.

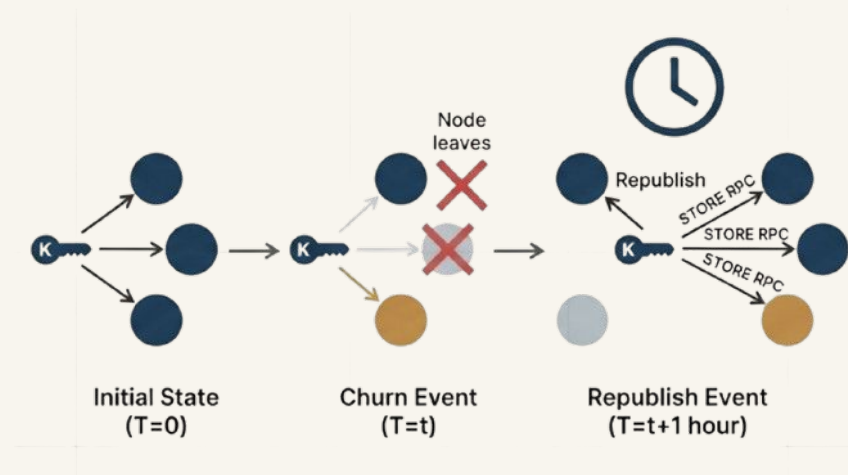
2. Self-Lookup: 'u' performs a 'node lookup' for its own ID. This populates its k-buckets and inserts itself into other nodes' tables.

3. 'u' refreshes buckets further away than its closest neighbour, ensuring a robust view of the network.

Ensuring Persistence Through Efficient Republishing

To counteract node churn and ensure data survival, $\langle \text{key}, \text{value} \rangle$ pairs must be periodically republished. Kademlia does this efficiently.

- **The Need:** Data is lost if 1. Original node leaves, or 2. new, closer nodes join.
- **The Solution:** Stored pairs expire and must be republished once per hour.
- **Possible Optimisations:**
 - Avoiding Storms: Only one of the k nodes republishes per hour.
 - Amortising Lookups: Bucket refreshes provide enough local knowledge to avoid a full lookup before every republish.
 - Cache implementation



THANK YOU